

CS3212 Algorithms

Aya Zirikly
10/01/2026

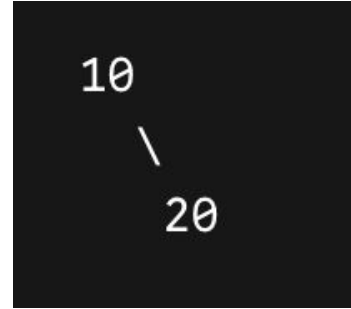
AVL trees

Balance Factor(x) = height(left_child) - height(right_child)

height(x) = 1 + max(height(x.left), height(x.right))

Step 1: Insert 10, 20, 30 (Triggers Right-Right Case)

Insert 10, then 20



Insert 30

```
10 (Balance = -2) [Unbalanced!]
```

```
\
```

```
20
```

```
\
```

```
30
```

Insert 30

Node 10 balance factor becomes **-2** (Left Height 0 – Right Height 2).

Let us look at height and BF for each node

```
10 (Balance = -2) [Unbalanced!]
```

```
\
```

```
20
```

```
\
```

```
30
```

Insert 30

Scenario A: height of empty tree[] is -1 and leaf node is 0

$h(30) = 0$ (leaf node)

$BF(30) = h(L) - h(R) = -1 - (-1) = 0$

$h(20) = 1 + \max(h(L), h(R)) = 1 + \max(h[], h(30)) = 1 + \max(-1, 0) = 1$

$BF(20) = h(L) - h(R) = -1 - (0) = -1$

$H(10) = 1 + \max(h[], h(20)) = 1 + \max(-1, 1) = 2$

$BF(10) = h(L) - h(R) = h(L) - h(20) = -1 - 1 = -2$

10 (Balance = -2) [Unbalanced!]



Insert 30

Node 10 balance factor becomes **-2** (Left Height 0 – Right Height 2).

Scenario B: height of empty tree is 0 and leaf node is 1

$h(30) = 1$ (leaf node)

$BF(30) = h(L) - h(R) = 0 - 0 = 0$

$h(20) = 1 + \max(h(L), h(R)) = 1 + \max(h[], h(30)) = 1 + \max(0, 1) = 2$

$BF(20) = h(L) - h(R) = 0 - 1 = -1$

$H(10) = 1 + \max([], h(20)) = 1 + \max(0, 2) = 3$

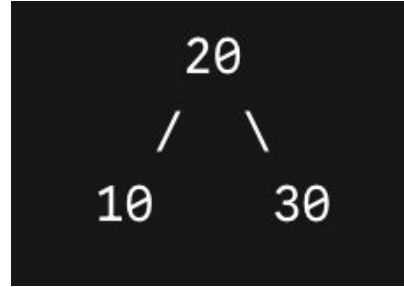
$BF(10) = h(L) - h(R) = h([]) - h(20) = 0 - 2 = -2$

10 (Balance = -2) [Unbalanced!]



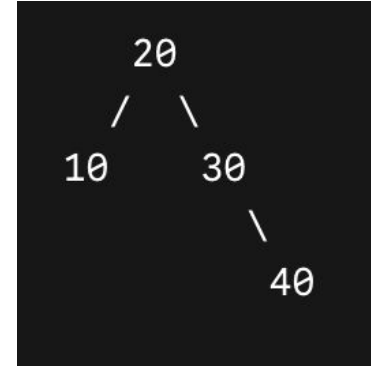
Fix: Left Rotation at 10

Rotate 20 up to become the new root.

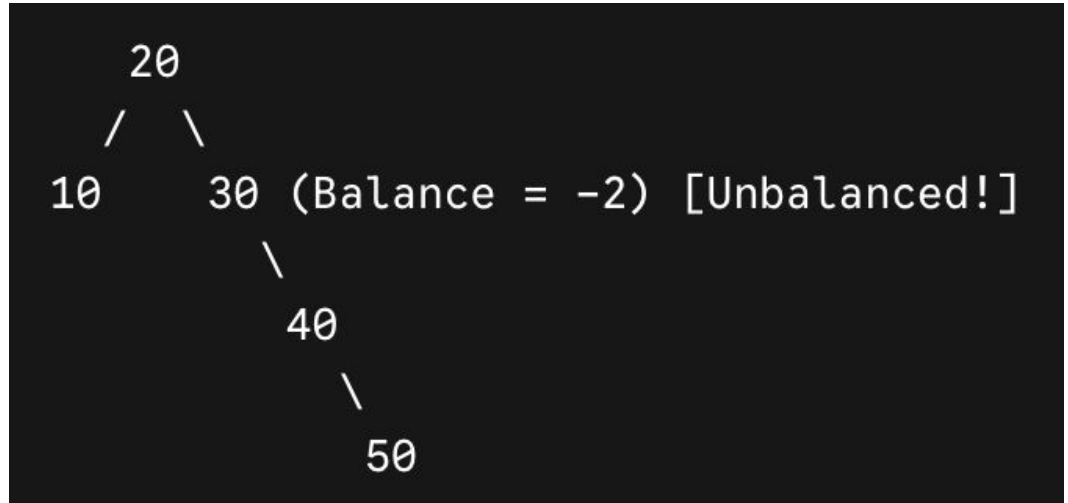


Step 2: Insert 40, 50 (Triggers Right-Right Case)

Insert 40



Insert 50



Node 30 balance factor becomes **-2** (Left Height 0 – Right Height 2).

Calculate height and BF for each node.

Insert 50

Scenario A: height of empty tree is -1 and leaf node is 0

$h(50) = 0$ (leaf node)

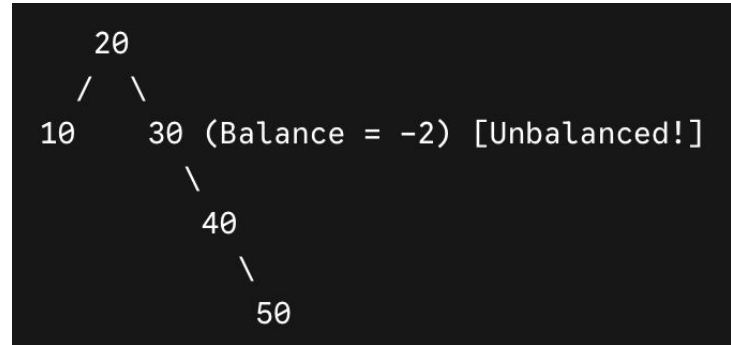
$BF(50) = -1 - (-1) = 0$

$h(40) = 1 + \max(h(L), h(R)) = 1 + \max(h[], h(50)) = 1 + \max(-1, 0) = 1$

$BF(40) = h[] - h(50) = -1 - 0 = -1$

$H(30) = 1 + \max([], h(40)) = 1 + \max(-1, 1) = 2$

$BF(30) = h(L) - h(R) = h[] - h(40) = -1 - 1 = -2$



Insert 50

Scenario B: height of empty tree is 0 and leaf node is 1

$h(50) = 1$ (leaf node)

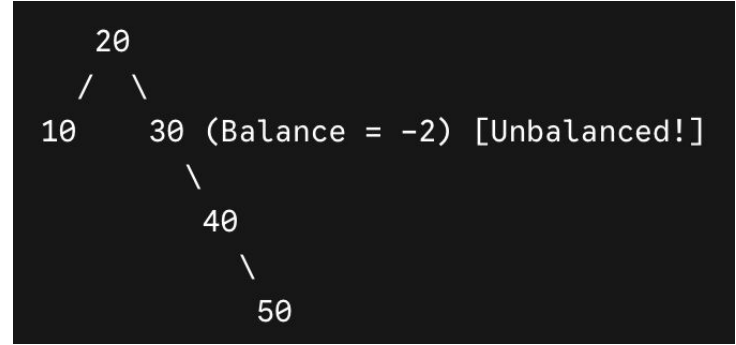
$BF(50) = h(\text{[]}) - h(\text{[]}) = 0 - 0 = 0$

$h(40) = 1 + \max(h(L), h(R)) = 1 + \max(h(\text{[]}), h(50)) = 1 + \max(0, 1) = 2$

$BF(40) = h(\text{[]}) - h(50) = 0 - 1 = -1$

$H(30) = 1 + \max(\text{[]}, h(40)) = 1 + \max(0, 2) = 3$

$BF(30) = h(L) - h(R) = h(\text{[]}) - h(40) = 0 - 2 = -2$



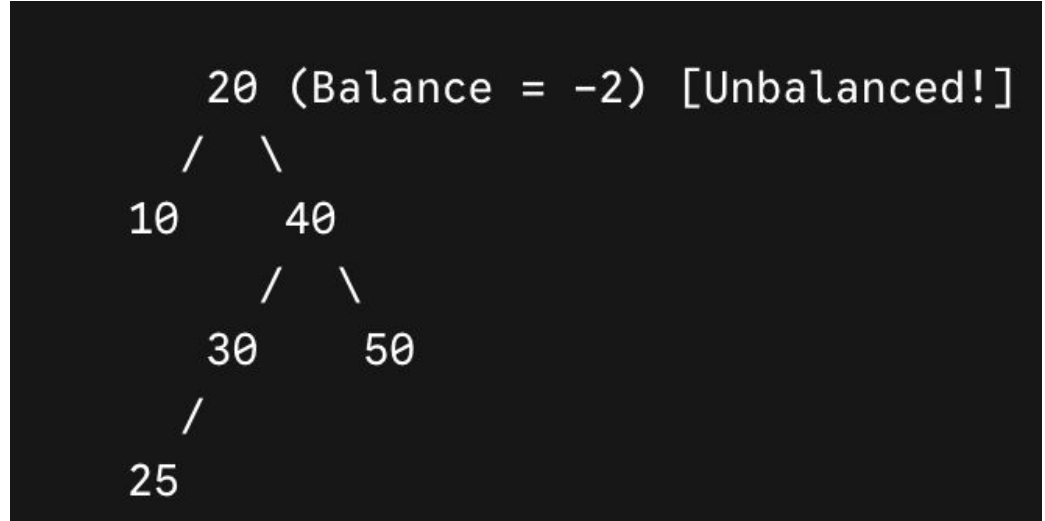
Insert 25 (Triggers Right-Left Case)

Insert 25

Walk down: $25 > 20 \rightarrow 25 < 40 \rightarrow 25 < 30$. Insert 25 as left child of 30.

Node 20 balance factor becomes -2 (Left Height 1 – Right Height 3).

Calculate height and BF for each node.

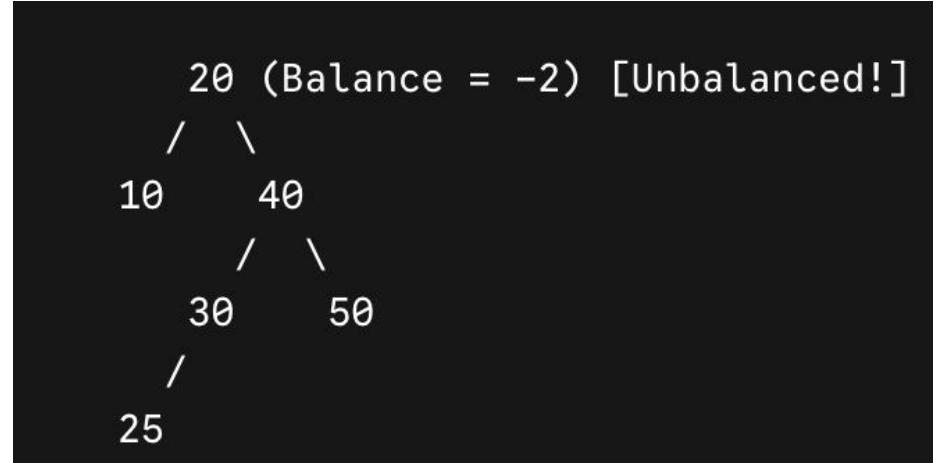


Insert 25 (Triggers Right-Left Case)

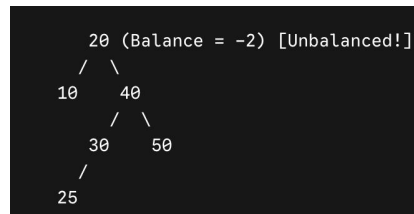
Insert 25

Walk down: $25 > 20 \rightarrow 25 < 40 \rightarrow 25 < 30$. Insert 25 as left child of 30.

Node 20 balance factor becomes **-2** (Left Height 1 – Right Height 3).



Because 25 was inserted into the **left** subtree of 40 and 40 is the **right** child of 20, this is a **Right-Left (RL) Case**.



Scenario A: height of empty tree is -1 and leaf node is 0

$h(50) = 0$ (leaf node)

$BF(50) = h(L) - h(R) = -1 - (-1) = 0$

$h(25) = 0$, $BF(25) = 0$ (leaf)

$h(10) = 0$, $BF(10) = 0$ (leaf)

$h(30) = 1 + \max(h(25), []) = 1 + \max(0, -1) = 1$

$BF(30) = h(25) - h([]) = 0 - (-1) = 1$

$h(40) = 1 + \max(h(30), h(50)) = 1 + \max(1, 0) = 2$

$BF(40) = h(30) - h(50) = 1 - 0 = 1$

$H(20) = 1 + \max(h(10), h(40)) = 1 + \max(0, 2) = 3$

$BF(20) = h(10) - h(40) = 0 - 2 = -2$

Scenario B: height of empty tree is 0 and leaf node is 1

$h(50) = 1$ (leaf node)

$BF(50) = h(L) - h(R) = 0 - 0 = 0$

$h(25) = 1$, $BF(25) = 0$ (leaf)

$h(10) = 1$, $BF(10) = 0$ (leaf)

$h(30) = 1 + \max(h(25), []) = 1 + \max(1, 0) = 2$

$BF(30) = h(25) - h([]) = 1 - 0 = 1$

$h(40) = 1 + \max(h(30), h(50)) = 1 + \max(2, 1) = 3$

$BF(40) = h(30) - h(50) = 2 - 1 = 1$

$H(20) = 1 + \max(h(10), h(40)) = 1 + \max(1, 3) = 4$

$BF(20) = h(10) - h(40) = 1 - 3 = -2$

Core Operations Overview (B-Tree / B+ Tree)

Search: Traverse down child pointers based on range comparisons inside each node.

Insertion & Node Splitting:

- Insert key into appropriate leaf.
- If node overflows (more than $m - 1$ keys), **split node into two** and push the middle key up to the parent.
- Splits propagate upwards (the tree grows *up*, not down).

Deletion & Node Merging:

- Remove key from leaf.
- If node underflows (fewer than $\text{ceiling}(m / 2) - 1$ keys), borrow keys from siblings or **merge** with a sibling.

B+ Tree —Insert

Leaf-First Insertion: New keys are always inserted directly into a leaf node at the bottom of the tree.

Bottom-Up Growth: B+ Trees grow upward from the leaves toward the root, which guarantees that all leaf nodes remain at the exact same depth.

Copy Up vs. Push Up Rule:

- ❑ Leaf Node Split -> COPY UP: The middle key is copied to the parent because leaf nodes must retain all actual data keys.
- ❑ Internal Node Split -> PUSH UP: The middle key is moved (promoted) to the parent because internal nodes only serve as routing index markers.

Step-by-Step Insertion Algorithm

① Locate the Leaf Page

Search phase

Traverse the tree from the root down to find the correct leaf node L where the key belongs in sorted order.

② Check Node Capacity

Evaluation phase

Count the keys currently in leaf L.

- **If space exists** ($\text{Keys} < m - 1$): Insert the key in sorted position. **Done.**
- **If leaf L is full** ($\text{Keys} = m - 1$): Proceed to Step 3 (Node Split).

③ Split the Leaf Node

Leaf overflow handling

Add the new key to L temporarily (making m total keys) and divide it into two leaf nodes:

- **Left Leaf:** Retains the first half of the keys.
- **Right Leaf:** Receives the remaining keys.
- **Copy Up:** Take the **first key of the Right Leaf** and copy it into the parent node as a routing key.

④ Handle Parent Cascading (If Needed)

Internal overflow handling

If inserting the copied routing key causes the parent internal node to overflow:

- Divide the internal node's keys and child pointers into two nodes.
- **Push Up:** Move the **exact middle key** up into its parent (do not leave a copy behind in the internal node).
- Repeat this step recursively up the tree if parents continue to overflow.

⑤ Split Root Node (Tree Height Increase)

Root overflow handling

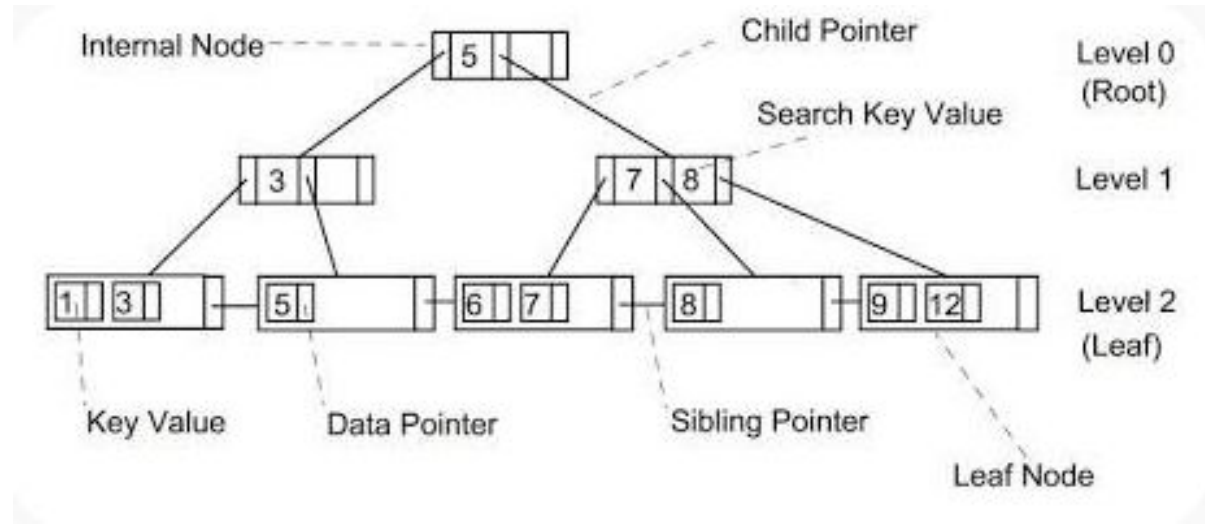
If the recursive push-up reaches the root node and it overflows, split the root into two internal nodes and **push up** the middle key to become a **new single-key root node**.

- *This is the only way a B+ Tree increases its height.*

Summary

Node Being Split	Action Taken on Middle/Separator Key	Does key remain in original node?
Leaf Node	Copy Up	YES (Retains key to preserve data)
Internal Node	Push Up (Promote)	NO (Moved up; removed from lower node)

Data structure



Implementation note: Although shown as a list for clarity, the keys in a node are stored in sorted order in an array-like layout: a contiguous array for fixed-length keys, or a slot (offset) directory for variable-length keys. This enables $O(\log m)$ binary search within the page (see Slide 32) and gives better cache locality than a linked structure. The trade-off is that insertions and deletions require shifting entries, which costs $O(m)$ per node.

Tracing example ($m = 3$, Max Keys = 2)

Tracing example ($m = 3$, Max Keys = 2)

Step 1: Insert 10 and 20

Insert 10 and 20 into the empty tree. Both fit inside a single leaf node in sorted order.

```
[ 10 | 20 ] <-- Single Leaf Node (Root)
```

Status: Leaf capacity is 2 keys. Node is full, but no overflow yet.

Step 2: Insert 30

1. Search for 30. It belongs in the leaf node [10 | 20].
2. Temporarily add 30: [10 | 20 | 30] -> **Overflow!** (3 keys exceeds max of 2).
3. **Split Leaf Node:**
 - **Left Leaf:** Retains [10]
 - **Right Leaf:** Receives [20 | 30]
4. **COPY UP Rule:** Take the first key of the right leaf (20) and copy it up to create a new parent root node.

```
      [ 20 ]                <-- Root (Internal Routing Node)
      /   \
 [ 10 ]   [ 20 | 30 ] <-- Leaf Nodes (Data)
```

Routing Verification:

- Left child contains keys strictly less than 20 (< 20).
- Right child contains keys greater than or equal to 20 (>= 20).

Step 3: Insert 40

Route from Root [20]: Compare 40 with root key 20. Since $40 \geq 20$, follow the right child pointer to leaf [20 | 30].

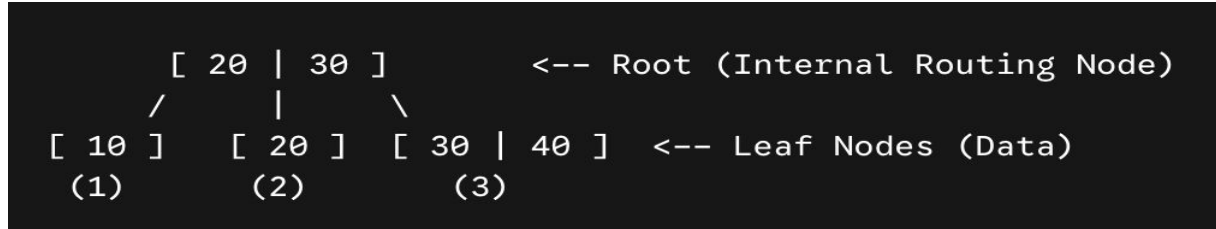
Temporarily add 40: [20 | 30 | 40] -> **Overflow!**

Split Leaf Node [20 | 30 | 40]:

- **Left Leaf:** Retains [20]
- **Right Leaf:** Receives [30 | 40]

COPY UP Rule: Take the first key of the right leaf (30) and copy it up to the parent root node.

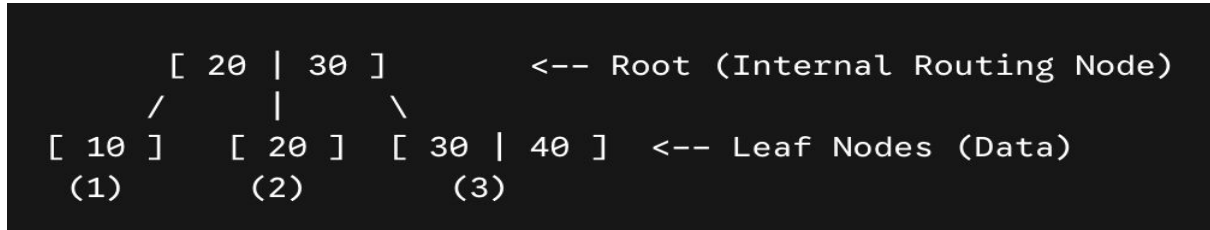
The parent root node [20] has space for one more key, so 30 is inserted directly into the root in sorted order.



Final Routing Verification:

- **Child (1):** Keys $< 20 \rightarrow [10]$
- **Child (2):** Keys ≥ 20 and $< 30 \rightarrow [20]$
- **Child (3):** Keys $\geq 30 \rightarrow [30 \mid 40]$

Leaf Linked List: All leaves are linked horizontally in sorted order: $[10] \leftrightarrow [20] \leftrightarrow [30 \mid 40]$.



Step 4: Insert 50

Sub-Step 4A:

1. **Route from Root [20 | 30]:**
 - Compare 50 against root keys 20 and 30.
 - Since $50 \geq 30$, follow the rightmost child pointer to leaf [30 | 40].
2. **Add 50:** Leaf becomes [30 | 40 | 50] -> **Overflow!** (3 keys exceeds max capacity of 2).

Sub-Step 4B

1. **Divide Leaf:**
 - **Left Leaf:** Retains [30]
 - **Right Leaf:** Receives [40 | 50]
2. **COPY UP Rule:** Take the first key of the right leaf (40) and copy it up to the parent root node.

Sub-Step 4C

Because the root node [20 | 30 | 40] now holds 3 keys, it overflows and must split!

1. **Divide Internal Keys:**

- Keys: 20, 30, 40
- Middle Key: 30

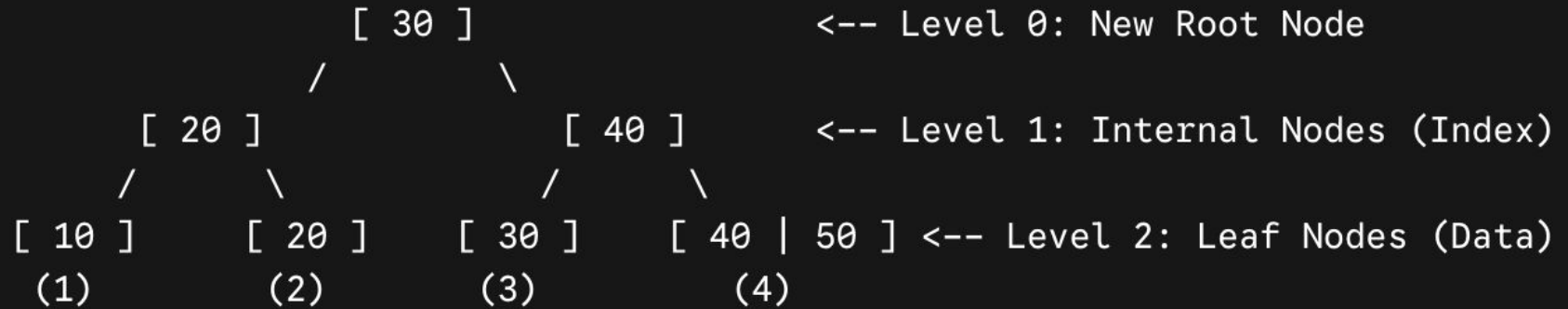
2. **PUSH UP Rule (Internal Nodes):**

- Move (promote) the middle key 30 UP to become the **new single-key root**.
- **Do NOT leave a copy of 30 behind** in the lower internal nodes.

3. **Reassign Child Pointers:**

- **Left Internal Node:** Retains key [20] with children pointing to leaves [10] and [20].
- **Right Internal Node:** Retains key [40] with children pointing to leaves [30] and [40 | 50].

Sub-Step 4C



Time complexity

Operation	Time Complexity	Primary Bottleneck
Search (Point Query)	$O(\log_m N)$	Disk I/O (Traversing tree height h)
Insert	$O(\log_m N)$	Disk I/O (Searching leaf + potential page splits)
Delete	$O(\log_m N)$	Disk I/O (Searching leaf + potential page merges)

- ★ N is the total number of records
- ★ m is the tree order (branching factor)

Key Insights

Base-m Logarithm Shrinks Tree Height: Because the branching factor m is very large (e.g., $m = 1000$), $\log_{1000}(N)$ is drastically smaller than a binary tree's $\log_2(N)$. For 1 billion records:

- **Binary Search Tree ($\log_2 N$):** ~30 disk reads
- **B+ Tree ($\log_m N$):** 3 disk reads

Node-Level Search vs. Tree Traversal: Searching inside a single node loaded in RAM takes $\log_2(m)$ steps using binary search. But because reading from disk takes milliseconds while reading from RAM takes nanoseconds, disk I/O completely dominates performance. We treat the in-RAM search as negligible, so total runtime is directly proportional to tree height: $O(\log_m N)$.

RAM (Fast, Volatile)

+-----+
Root Node (Pinned)
Top Internal Nodes (Cached)
"Hot" Leaf Pages (Cached)
Dirty Buffer Pages
+-----+

^

| Operations take Nanoseconds

DISK (Slow, Persistent)

+-----+
Full B+ Tree Structure
- Root Page
- All Internal Pages
- All Leaf Data Pages
+-----+

^

| Operations take Milliseconds

Dynamic Self-Adjusting Binary Search Trees: Splay Trees

What if a data structure automatically learned which data you use most, without storing a single byte of height or balance metadata?

Why

- ❑ **The 80/20 Rule (Pareto Principle):** In real-world software (caches, routers, databases), 80% of access requests target 20% of the data (*temporal locality*).
- ❑ **The Problem with Rigid Trees:** AVL and Red-Black trees keep every path at $O(\log N)$ regardless of access frequency—a hot item at depth $\log N$ always costs $\log N$ to reach.
- ❑ **The Goal:** Build an adaptive tree where frequently accessed items naturally drift to the top, yielding $O(1)$ amortized lookup for "hot" data.

What is a splay tree

Definition: A self-adjusting Binary Search Tree created by Sleator and Tarjan (1985).

Zero Metadata Overhead: No height balance factors (AVL) or color bits (Red-Black) stored in nodes.

The Core Mechanism — Splaying: Every operation (search, insert, delete) finishes by moving the accessed node to the root via a series of tree rotations.

Splay Trees Properties

- ❑ Splay Trees follow standard BST Ordering
- ❑ In a Splay Tree, for any node with key K :
 - ❑ Every key in its left subtree is strictly smaller ($< K$).
 - ❑ Every key in its right subtree is strictly larger ($> K$).
- ❑ Rotations (Zig, Zig-Zig, Zig-Zag) never break this order. They simply change the parent-child relationships to bring target keys closer to the root while maintaining valid BST properties.

Splay Trees vs. B+ Trees

Feature	B+ Trees	Splay Trees
Primary Medium	Disk / Secondary Storage	Main Memory (RAM)
Structural Invariant	Rigid height balance (all leaves at depth h)	Dynamic BST shape (no strict height bound)
Data Placement	Exclusively in leaf nodes (internal nodes = index directory)	Data stored in every node (internal and root)
Adaptability	Uniform access cost for all keys	Fast $O(1)$ access for heavily biased/hot data

Python

```
class SplayNode:
    def __init__(self, key):
        self.key = key          # Payload / Data value
        self.left = None       # Reference to left child
        self.right = None      # Reference to right child
        self.parent = None     # Reference to parent (essential for Zig/Zag rotation)

class SplayTree:
    def __init__(self):
        self.root = None      # Points to the current root node
```

C++

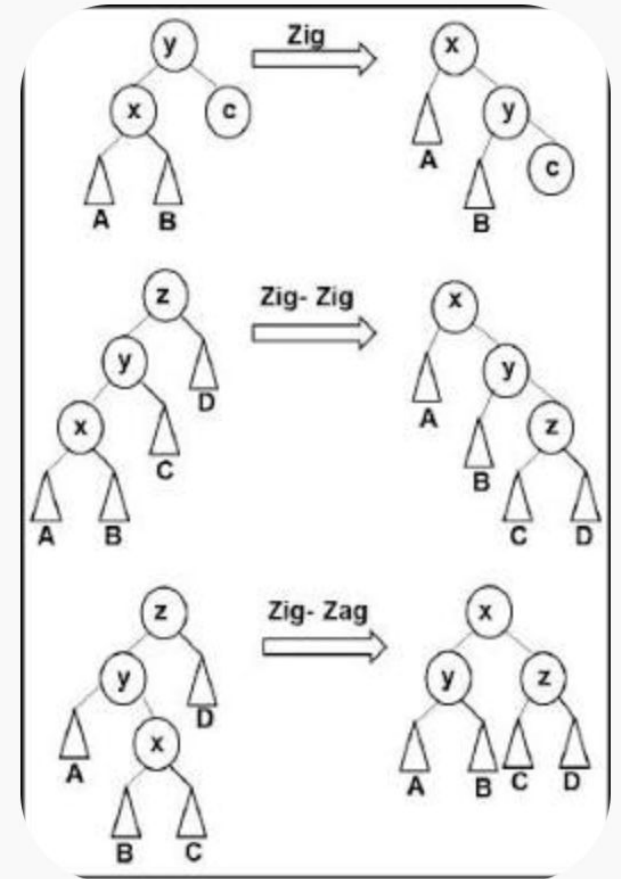
```
struct SplayNode {
    int key;          // Payload/Data
    SplayNode* left;  // Pointer to left child
    SplayNode* right; // Pointer to right child
    SplayNode* parent; // Pointer to parent (essential for Zig/Zag rotations)

    SplayNode(int val)
        : key(val), left(nullptr), right(nullptr), parent(nullptr) {}
};
```

The 3 Splaying Cases

- ❑ Whenever a node x is searched, inserted, or accessed, it is pushed all the way to the top of the tree to become the new root.
- ❑ To move x up, you evaluate its position relative to its parent p and grandparent g , applying one of these three cases repeatedly until x becomes the root.

Goal: Push target node x to the root using x 's parent (p) and grandparent (g).



The 3 primary Splay Tree rotation cases. Source: ResearchGate

The 3 Splaying Cases

Case 1: Zig (Single Rotation)

- **When it applies:** Node x is one step away from the root. Its parent p is the root, so there is no grandparent g .
- **Mechanism:** Perform a single standard binary search tree rotation.
 - If x is a left child, perform a **Right Rotation** around p .
 - If x is a right child, perform a **Left Rotation** around p .
- **Purpose:** This is always the **terminal step** of a splay sequence when x starts at an odd depth.

The 3 Splaying Cases

Case 2: Zig-Zig (Double Rotation — Straight Line)

- **When it applies:** Node x , its parent p , and its grandparent g form a straight line:
 - Left-Left chain: x is a left child of p , and p is a left child of g .
 - Right-Right chain: x is a right child of p , and p is a right child of g .
- **Mechanism:** Rotate parent p around grandparent g **FIRST**, then rotate x around parent p .
- **Why the order matters:** Rotating p around g first is the crucial mechanism that **halves the depth of the entire path**. If you naively rotated x around p twice, you would just move x to the top while pushing the rest of the deep path down into another narrow line.

The 3 Splaying Cases

Case 3: Zig-Zag (Double Rotation — Bent/Elbow Shape)

- **When it applies:** Node x , parent p , and grandparent g form a zig-zag (bent) path:
 1. Left-Right shape: x is a right child of p , and p is a left child of g .
 2. Right-Left shape: x is a left child of p , and p is a right child of g .
- **Mechanism:** Perform two standard AVL-style rotations centered on x :
 1. Rotate x around p .
 2. Rotate x around g .
- **Outcome:** Node x becomes the local parent, with p and g becoming its left and right children.

The 3 Splaying Cases

- Case 1: Zig (Single Rotation) Condition: p is the root (no grandparent g). Action: Perform a single BST rotation (left or right) to make x the root.
- Case 2: Zig-Zig (Double Rotation — Same Direction) Condition: x and p are both left children, or both right children. Action: Rotate p around g first, then rotate x around p . Teaching Note: Emphasize that rotating p first is what halves the height of the tree over time! Doing two single rotations (Zig-Zag style) wouldn't flatten deep paths.
- Case 3: Zig-Zag (Double Rotation — Opposite Directions) Condition: x is a right child and p is a left child (or vice versa). Action: Standard AVL-style double rotation: rotate x around p , then rotate x around g .

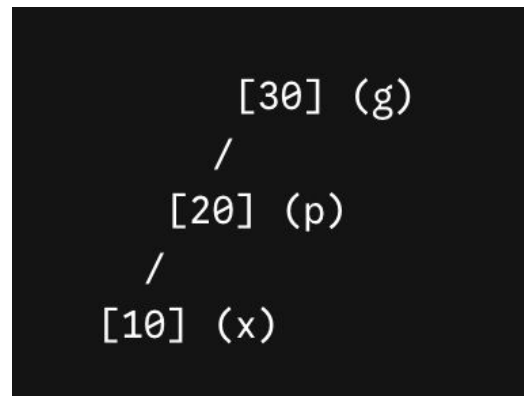
Tracing example

Search for key **10** in a tree where keys were inserted in descending order (30, 20, 10):

- **Target node (x):** 10
- **Parent (p):** 20
- **Grandparent (g):** 30

Because both x (10) and p (20) are **left children** (a straight line)

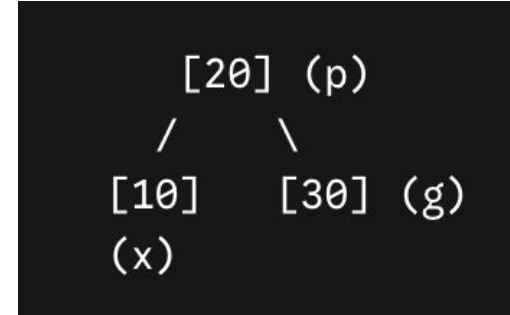
=> **Zig-Zig** condition.



Rotate Parent (p) Around Grandparent (g)

Perform a **Right Rotation** on p (20) around g (30).

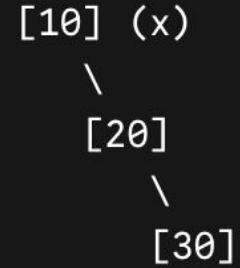
Result: 20 moves up to become local root; 30 becomes its right child.



Rotate Target (x) around Parent (p)

Action: Perform a **Right Rotation** on x (10) around p (20).

Result: 10 moves up to the root; 20 becomes its right child.



The Zig-Zag Transformation (Rebalances Elbows): example

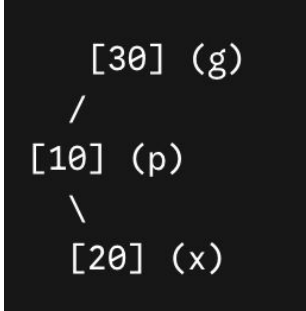
Goal: Accessing target key **20** in a bent subtree path.

Target (x): 20

Parent (p): 10

Grandparent (g): 30

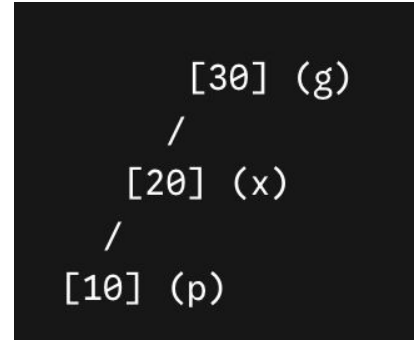
Condition Check: p (10) is a left child, but x (20) is a right child -> **Zig-Zag Case (Left-Right Elbow)**.



Rotate Target (x) around Parent (p)

Perform a **Left Rotation** on x (20) around p (10).

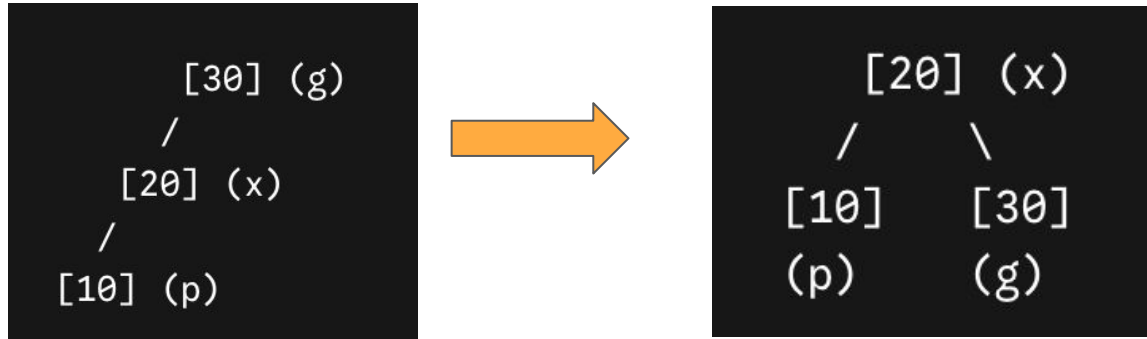
Result: 20 moves up to replace 10, turning the bent path into a straight left-chain.



Rotate Target (x) around Grandparent (g)

Perform a **Right Rotation** on x (20) around g (30).

20 moves up to become the new root of this subtree, with 10 as its left child and 30 as its right child.



Primary Tree Operations

Search(k): Standard BST lookup + splay target node (or last visited node if not found) to root.

Insert(k): Standard BST placement + splay newly inserted node k to root.

Delete(k):

1. Search for k (splays k to root).
2. Drop root k. (Leaves left subtree L and right subtree R).
3. Find maximum element in L and splay it to L's root.
4. Attach R as the right child of L's root.